Requirements Document: TRACE

Software Bug Tracking & Quality Assurance System

Version: v0.1.0
Date: October 18, 2026
Course: Software Construction and Development (SCD) - IMSCIENCES

1. Project Team Details

- **Team Member 1:** Muhammad Ihtesham (GitHub: ihtesham267)
- **Team Member 2:** Atizaz shah (GitHub: atizazmalik91)
- **Team Member 3:** Ghazal khan(GitHub: ghazalk350)

2. Problem Statement

In many software development projects, bug reporting and quality assurance tasks are scattered across emails, spreadsheets, and informal chat messages. This fragmented approach leads to lost bug reports, unclear ownership of issues, difficulty tracking the status of fixes, and a lack of centralized visibility into the overall health of the software. TRACE aims to solve this by providing a unified, centralized platform dedicated to tracking software defects from initial discovery to final resolution.

3. Objectives and Intended Users

Project Objectives:

- To develop a centralized platform for reporting, assigning, and tracking software bugs.
- To streamline communication between developers and QA testers.
- To provide real-time visibility into project health and bug resolution progress through a centralized dashboard.

Intended Users & Roles:

1. **Admin / Project Manager:** Can oversee the entire project, manage users, and assign reported bugs to specific developers.
2. **Developer:** Can view bugs assigned to them, update bug statuses (e.g., "In Progress", "Fixed"), and add technical remarks.
3. **Tester / QA:** Can report new bugs (with severity, priority, and reproduction steps), verify developer fixes, and close or reopen bugs based on re-testing.

4. Functional Requirements (FR)

ID	Requirement Description
FR-01	The system shall provide secure user authentication and role-based access control (Admin, Developer, Tester).
FR-02	The system shall allow a Tester to submit a detailed bug report including title, description, severity, priority, steps to reproduce, and expected vs. actual results.
FR-03	The system shall allow an Admin/Project Manager to assign an open bug to a specific Developer.
FR-04	The system shall allow a Developer to change the status of assigned bugs (e.g., Open → In Progress → Fixed).
FR-05	The system shall allow a Tester to verify a "Fixed" bug and change its status to either "Closed" (if resolved) or "Reopened"
FR-06	The system shall allow authorized users to leave comments or remarks on a bug ticket for cross-team communication.
FR-07	The system shall display a dashboard summarizing bug statistics (e.g., total open bugs, bugs by severity, bugs assigned per developer).

5. Non-Functional Requirements (NFR)

ID	Requirement Description
NFR-01	**Security:** User passwords must be securely hashed. Environment variables and database secrets must not be exposed in the GitHub repository.
NFR-02	**Usability:** The web interface must be intuitive, allowing users to report or update a bug in fewer than 3 clicks from the main dashboard.
NFR-03	**Maintainability:** The backend must be built using modular Python/Flask architecture to allow for easy future enhancements.
NFR-04	**Reliability:** The system must gracefully handle invalid data inputs (e.g., empty bug descriptions) using proper form validation without crashing.

6. Prioritized Features / Product Backlog This backlog outlines the planned features in order of development priority.

ID	Feature / Task	Priority	Status
T-01	Repository setup, documentation, and versioning	High	In Progress
T-02	Database schema design and Supabase integration	High	Backlog
T-03	User authentication and role management module	High	Backlog
T-04	Bug reporting and assignment functionality	High	Backlog
T-05	Bug lifecycle tracking (status updates & verification)	High	Backlog
T-06	Dashboard for bug statistics and project health	Medium	Backlog
T-07	Commenting system for individual bug tickets	Low	Backlog

7. Assumptions, Constraints, and Limitations

Assumptions:

- All users will access the system through modern web browsers.
- The development team has consistent internet access to interact with GitHub and the Supabase backend.

Constraints:

- **Technology Stack:** The application is strictly constrained to Python, Flask, Supabase (Database), HTML/CSS/JavaScript, and PyTest.
- **Timeline:** The project must be completed within the current semester's academic deadlines.

Known Limitations (v1.0):

- The initial version will not include automated email notifications for bug assignments or status changes.
- Advanced analytics (e.g., average time to resolve a bug) are excluded from the current scope.
- CI/CD (Continuous Integration/Continuous Deployment) pipelines are not part of this initial assignment phase.